DeliveryBrief AI Collaboration Note

Prepared for Elvis's review | 8 September 2026

Purpose

This log records how I used AI during the Quest and how I checked its work. I remain responsible for the workflow choice, technical decisions, code, evaluation, claims, and submission.

Collaboration entries

7 September 2026 Quest interpretation and scope

Tool and model. Codex coding agent.

Delegated work. I asked the agent to extract the Quest requirements, compare project directions, and turn the scoring rubric into a build strategy.

Accepted. I retained the recommendation to choose one recurring workflow with a measurable baseline, human approval, two integrations, and explicit failure cases.

Rejected or corrected. I did not accept the idea that a polished live demo alone would prove value. The plan now separates sample behavior from real-user evidence. I also changed the model-provider recommendation after confirming that I already had Anthropic API credit.

Verification. I checked the recommendation against the supplied Quest text and the Applied AI Engineer role description.

Decision I owned. I selected weekly delivery updates, a delivery manager as the user, GitHub and Google Docs as sources, a public demo, and Anthropic as the funded provider.

Artifact. docs/decisions.md and the implementation plan.

7 September 2026 System implementation

Tool and model. Codex coding agent.

Delegated work. I asked the agent to scaffold the Python application, typed schemas, source adapters, validation rules, storage, exports, evaluation fixtures, tests, and document drafts.

Accepted. The implementation uses a credential-free demo path and separately configured live adapters. Every factual report item carries evidence IDs. External sending remains outside the system.

Rejected or corrected. No field metric, real-user quote, or adoption claim was generated. The documents include an evidence checklist instead of invented results.

Verification. Static checks, automated tests, an evaluation run, application smoke testing, and rendered-document inspection must be recorded after execution.

Decision I owned. I will personally review the generated code, run the system with my credentials, conduct the user session, and approve every final claim.

Artifact. Repository implementation and docs/evidence-needed.md.

7 September 2026 Deployment and Day 1 evidence planning

Tool and model. Codex coding agent.

Delegated work. I asked the agent to verify the deployed Streamlit URL and break the Day 1 evidence work into practical next steps.

Accepted. The app URL is recorded as https://deliverybrief.streamlit.app/. The Quest acceptance time is recorded as Monday, 7 September 2026 at 3:00 PM WAT. Erwin's reply confirmed that the timer is active.

Rejected or corrected. The deployed URL was not treated as final because a logged-out check redirected to Streamlit sign-in. I need to make the app public before submitting it as the Working System link.

Verification. The deployed URL was checked without an existing logged-in browser session and returned a sign-in redirect instead of the public application.

Decision I owned. I will collect real Day 1 evidence manually and only include user feedback, timing numbers, and examples that I can explain and defend.

Artifact. docs/quest-control.md.

7 September 2026 Target-user perspective cleanup

Tool and model. Codex coding agent.

Delegated work. I gave the agent rough answers from the viewpoint of a project/product manager who sends weekly updates to an MD. I asked it to make the answers clearer and easier to use as Day 1 evidence.

Accepted. The cleaned note states the workflow, sources, bottleneck, trust conditions, rejection conditions, and examples of information that should not reach a client or MD.

Rejected or corrected. I did not present this as a fake external interview. It is recorded as Elvis-provided target-user perspective, with a note that a separate external interview is still useful if time allows.

Verification. I checked that the note reflects the supplied answers: weekly MD updates, GitHub PRs, developer notes, vague commits, document quality, context handling, consistency, and formatting.

Decision I owned. I chose to use this as Day 1 proxy evidence while keeping the limitation visible.

Artifact. evidence/day-1/interview-notes-elvis-proxy.md.

7 September 2026 Weekly example cleanup

Tool and model. Codex coding agent.

Delegated work. I supplied three public-safe weekly examples covering payment retry risk, cross-repository contract coordination, and reporting-pipeline migration/revert context. I asked the agent whether they were sufficient and how to use them.

Accepted. The examples were recorded as anonymized reconstructed weeks, with manual time estimates of 55 minutes, 75 minutes, and 40 minutes. The shared pattern is that Git history alone does not explain the real delivery state.

Rejected or corrected. I did not claim the examples are public PRs or exact production records. They are documented as reconstructed examples from real operating patterns.

Verification. I reviewed that each example includes what happened, PR information, developer notes, MD-facing summary, action items, blockers, manual time estimate, and private information removed.

Decision I owned. I chose to use reconstructed examples because public PRs were not available and private project material should not be exposed in a public Quest submission.

Artifact. evidence/day-1/weekly-examples-index.md and evidence/day-1/baseline-log.csv.

7 September 2026 Day 1 evaluation smoke test

Tool and model. Codex coding agent and deterministic demo generator.

Delegated work. I asked the agent to turn the three reconstructed weeks into a separate Day 1 evaluation dataset and run the current demo generator against it.

Accepted. The Day 1 dataset was created separately from the official ten-case suite. The smoke test result was recorded even though it failed all three cases.

Rejected or corrected. I did not hide the failure. The result shows that the demo generator cites evidence correctly but does not reliably split multiple action owners from one developer note.

Verification. I reviewed the executed result: grounding, coverage, and safety were 100% on all three cases, while action-item accuracy was 50% on all three cases.

Decision I owned. I will use this as a baseline limitation and compare it against Claude structured-output runs later.

Artifact. evaluation/day1/cases.json, evaluation/results/day1-demo.json, and evidence/day-1/day-1-summary.md.

7 September 2026 Anthropic Day 1 Haiku run and cost cap

Tool and model. Codex coding agent, Anthropic Claude Haiku 4.5, and local evaluation command.

Delegated work. I asked the agent to run the Day 1 reconstructed examples through Anthropic after adding my API key locally.

Accepted. The agent confirmed only that the key was present, without printing it. Haiku passed all three Day 1 reconstructed cases. The result was saved as evaluation/results/day1-haiku.json.

Rejected or corrected. The first Anthropic call exposed a schema compatibility bug: the structured-output schema needed explicit additionalProperties: false. I accepted the code fix and test for that. I also stopped further Sonnet reruns after deciding to cap cost.

Verification. Tests, lint, and mypy passed after the schema/evaluator fixes. The Haiku run recorded 3 of 3 cases passed with 100% grounding, coverage, action accuracy, and safety on this small Day 1 dataset.

Decision I owned. I chose Haiku as the default evaluation model for now because it passed the Day 1 examples at low cost. Sonnet should only be run again with explicit approval for a named comparison.

Artifact. deliverybrief/generator.py, deliverybrief/evaluation.py, tests/test_generator.py, tests/test_evaluation_workflow.py, evaluation/results/day1-haiku.json, and evidence/day-2/model-cost-control.md.

7 September 2026 Cost-estimation controls

Tool and model. Codex coding agent.

Delegated work. I asked whether the evaluation cost could be checked before running paid model calls.

Accepted. The evaluation command now supports --estimate-only for a zero-cost local estimate and --max-estimated-cost-usd to stop before calling Anthropic when the estimate is above the approved cap.

Rejected or corrected. I did not rely on memory or manual discipline as the only cost control. The estimator and cap are implemented in code and covered by tests.

Verification. The Day 1 Haiku estimate ran without an API call. The full Haiku estimate ran without an API call. A test command with a $0.01 cap stopped before any paid run. Pytest, Ruff, and mypy passed.

Decision I owned. I chose to require an estimate before future paid evaluation runs and to keep Sonnet gated behind explicit approval.

Artifact. deliverybrief/evaluation.py, deliverybrief/generator.py, tests/test_evaluation_workflow.py, tests/test_generator.py, and evidence/day-2/model-cost-control.md.

7 September 2026 Full Haiku evaluation and regression fixes

Tool and model. Codex coding agent, local test suite, and Anthropic Claude Haiku 4.5.

Delegated work. I approved running the full Haiku evaluation with a $0.30 cap after reviewing the local estimate.

Accepted. The final full Haiku result passed 9 of 9 scored report cases. The remaining transient API case is covered by an automated integration test. The result cost about $0.025316 for the nine paid cases.

Rejected or corrected. I did not accept the first 8-of-9 result as final. The failing cases led to two fixes: important evidence can no longer remain only in the executive summary, and action-like evidence is now flagged when no action owner or due date appears in the draft.

Verification. Focused regressions for case 03 and case 06 passed. The final full Haiku run passed. Pytest, Ruff, and mypy passed after the fixes.

Decision I owned. I kept Haiku as the default model because it passed the current full suite at low cost. I kept Sonnet disabled unless a specific comparison is approved later.

Artifact. evaluation/results/full-haiku.json, evidence/day-2/full-haiku-evaluation-summary.md, deliverybrief/generator.py, deliverybrief/evaluation.py, and deliverybrief/validator.py.

7 September 2026 Public app-run evidence

Tool and model. Streamlit public app, deterministic demo generator, and Codex coding agent.

Delegated work. I ran the public app in the browser, approved the generated sample report, and saved the exported files and screenshot. I asked the agent to organize the evidence.

Accepted. The app-run artifacts were copied into evidence/day-2/app-run/ and summarized as product-demo evidence.

Rejected or corrected. I did not treat the sample app run as proof of real-world time savings. It is evidence that the deployed workflow can be opened, approved, and exported.

Verification. The exported run summary shows status: approved, evidence_count: 5, edits_made: true, and no external sending. The screenshot shows the generated client-email preview with evidence IDs.

Decision I owned. I chose to use this evidence for the Working System and Case Study while keeping the time-savings claim separate.

Artifact. evidence/day-2/app-run-summary.md and files under evidence/day-2/app-run/.

7 September 2026 Direct-prompt baseline

Tool and model. Codex coding agent and Anthropic Claude Haiku 4.5.

Delegated work. I asked the agent to run a direct-prompt baseline after the capped DeliveryBrief Haiku evaluation passed.

Accepted. The direct baseline ran with a $0.07 cap and saved results to evaluation/results/direct-prompt-haiku.json. It cost about $0.007295 and passed 1 of 9 scored report cases under the automated audit.

Rejected or corrected. I did not describe the result as proof that Haiku cannot write useful prose. The result is documented as an auditability gap: direct prompting lacks stable evidence IDs, structured output, deterministic validation, approval controls, and export records.

Verification. The result file includes per-case excerpts, scoring details, and estimated cost. The comparison was added to the Evaluation Package and Case Study sources.

Decision I owned. I chose to use this result to explain why DeliveryBrief adds workflow structure around the model instead of only calling Claude.

Artifact. scripts/run_direct_prompt_baseline.py, evaluation/results/direct-prompt-haiku.json, and evidence/day-2/direct-prompt-baseline-summary.md.

Daily continuation format

For every later use, add the date, task, tool and model, delegated work, accepted output, rejected or corrected output, verification, personal decision, and artifact reference. Link corrections to commits, tests, or result files where possible.

8 September 2026 Reliability revision

Tool and model: Codex coding agent in this task; no new Anthropic API requests. Elvis requested messy samples, tests, documented decisions, client documents, and cost controls.

Delegated: shared approval checks, evidence snapshots, session separation, input validation, connector pagination, 48 named cases, extra boundary/UI tests, document exports, and documentation updates.

Accepted in the implementation: the Python/Streamlit architecture, conservative paid-request reservations, five free scenarios, and human-controlled approval.

Corrected through inspection and tests: citation validity was mislabeled as grounding; storage approval bypassed UI validation; the phone detector damaged timestamps; “No completed work” triggered completion. These failures are recorded in the reliability log.

Verification: executed pytest output feeds evaluation/results/reliability-v2.json. Static checks, browser verification, and rendered-document inspection support the handoff. Automated checks are not user feedback or a live Anthropic benchmark.

Elvis's decisions: client updates stay primary, the public demo stays free, and additional paid model calls require approval. Elvis's final code walkthrough, factual review, and consenting-user observation are still pending. This entry does not claim they happened.

Artifacts: docs/reliability-upgrade.md, the expanded tests, and the revised application.

8 September 2026 Maintainability refactor

Tool and model: Codex coding agent in this task; no Anthropic API requests.

Delegated: separate the working prototype into clearer runtime layers while preserving the existing Streamlit entrypoint and old Python imports.

Accepted in the implementation: generation now lives under deliverybrief/generation/, approval under deliverybrief/approval/, SQLite under deliverybrief/persistence/, exports under deliverybrief/exporting/, workflow orchestration under deliverybrief/services/, and the Streamlit UI under deliverybrief/ui/.

Corrected through inspection and tests: moving Streamlit UI code exposed Python module-caching behavior in the test runner, so the root app.py now imports or reloads the UI module explicitly. Several scripts were changed so importing them does not accidentally execute work.

Verification: the refactor added structure tests and kept the full automated suite passing locally. Ruff, mypy, pytest, and the secret scanner were run after the move. After push, GitHub Actions exposed a Linux import-path issue in the new structure test. I corrected the test to import helper scripts by file path and the final CI run 34189560471 passed.

Elvis's decisions: review the refactor before merging, then keep the public demo on the verified main branch once local checks and GitHub Actions passed.

Artifacts: docs/developer-architecture.md, tests/test_project_structure.py, and the refactored package folders.